Monte-Carlo Project - Swing Contract

Thomas HANUS and Aurélien PEREZ

Abstract

This project focuses on swing contracts and their pricing. We introduce the theoretical stochastic optimal control framework used for pricing and hedging these products, and we study a parametric method—both with and without neural networks—to efficiently solve the associated optimization problem. Particular attention is given to evaluating the performance of the proposed methods, as well as analyzing the effectiveness of the optimization algorithms used to determine the optimal set of parameters.

1 Introduction

Swing contracts, commonly used in energy markets, allow buyers to purchase variable amounts of energy at fixed dates, subject to certain constraints. Valuing these contracts is a complex task that can be formulated as a Stochastic Optimal Control problem. Two main approaches are typically employed:

Backward Dynamic Programming Principle : This method uses dynamic programming equations to recursively compute the optimal value. However, it often suffers from high computational cost and memory usage, especially in high-dimensional settings.

Global Optimization: This approach formulates the problem as a stochastic optimization task, applying Stochastic Gradient Descent to Monte Carlo simulations. The optimal control strategy is approximated using parameterized functions.

In the paper Swing Contract Pricing: With and Without Neural Networks Lemaire, Pagès, and Yeo (2024), the authors introduce two families of parameterized functions: one based on analytical insights into the optimal control, and another using an informed deep neural network that incorporates the same prior knowledge. These functions are optimized using both Adam and Preconditioned Stochastic Gradient Langevin Dynamics (PSGLD). The results demonstrate that PSGLD enhances both the speed and accuracy of the optimization process compared to existing state-of-the-art techniques.

2 Swing options pricing theory

2.1 Mathematical description of the contract

A **swing option** is a derivative that gives the holder the **right to purchase variable amounts of energy** at specific *exercise dates*, denoted by $t_\ell = \frac{\ell T}{n}$, for $\ell = 0, \ldots, n-1$, until maturity T. At each date, energy is bought at a fixed strike price K.

2.1.1 Volume Constraints

Two types of constraints apply:

- **Local constraints**: at each date t_ℓ, the volume q_ℓ must satisfy:

$$0 \leq q_{\min} \leq q_\ell \leq q_{\max} \tag{2.1}$$

- **Global constraints**: the total volume purchased over the life of the contract must lie within:

$$Q_n = \sum_{\ell=0}^{n-1} q_\ell \in [Q_{\min}, Q_{\max}] \tag{2.2}$$

2.1.2 Physical Space of the Option

Based on local and global constraints, we define the **physical space** of the swing option, which describes the admissible cumulative consumptions at each exercise date. Two bounding functions are used:

- Lower bound:

$$Q_{\text{down}}(t_\ell) = \max\left(0, Q_{\min} - (n-\ell) \cdot q_{\max}\right) \tag{2.3}$$

- Upper bound:

$$Q_{\text{up}}(t_\ell) = \min\left(\ell \cdot q_{\max}, Q_{\max}\right) \tag{2.4}$$

At maturity t_n, the bounds satisfy $Q_{\text{down}}(t_n) = Q_{\min}$ and $Q_{\text{up}}(t_n) = Q_{\max}$.

2.1.3 Decision Making at Each Exercise Date

If the cumulative consumption at time t_ℓ is Q_ℓ, then at $t_{\ell+1}$, the total consumption must lie within $[L_{\ell+1}(Q_\ell), U_{\ell+1}(Q_\ell)]$.

where:

$$L_{\ell+1}(Q_\ell) = \max(Q_{\text{down}}(t_{\ell+1}), Q_\ell + q_{\min}) \tag{2.5}$$

$$U_{\ell+1}(Q_\ell) = \min(Q_{\text{up}}(t_{\ell+1}), Q_\ell + q_{\max}) \tag{2.6}$$

The admissible purchase volume at t_ℓ therefore lies within:

$$[A_\ell^-(Q_\ell), A_\ell^+(Q_\ell)] = [L_{\ell+1}(Q_\ell) - Q_\ell, U_{\ell+1}(Q_\ell) - Q_\ell] \tag{2.7}$$

2.2 Pricing

2.2.1 Formulation as a stochastic optimal control problem

We consider a filtered probability space:

$$(\Omega, \mathcal{F}, \{\mathcal{F}_{t_\ell}\}_{0\le\ell\le n-1}, \mathbb{P}),$$

where $\mathcal{F}_{t_\ell}$ is the filtration of a price process $(S_{t_\ell})_{0\le\ell\le n-1}$. The decision process $(q_\ell)_{0\le\ell\le n-1}$ is $\mathcal{F}_{t_\ell}$-adapted, and given cumulative consumption $Q \in \mathbb{R}_+$, an admissible strategy from time t_k onward must satisfy:

$$\begin{aligned}\mathcal{A}_{k,Q}^{Q_{\min},Q_{\max}} &= \left\{ (q_\ell)_{k\le\ell\le n-1} \mid q_\ell \in [q_{\min}, q_{\max}],\ \sum_{\ell=k}^{n-1} q_\ell \in \left[(Q_{\min}-Q)^+, Q_{\max}-Q\right] \right\} \\ &= \left\{ (q_\ell)_{k\le\ell\le n-1} \mid q_\ell \in [A_\ell^-(Q_\ell), A_\ell^+(Q_\ell)], \quad \text{where} \quad Q_\ell = Q + \sum_{i=k}^{\ell-1} q_i \right\}. \end{aligned} \tag{2.8}$$

At each exercise date t_ℓ, purchasing q_ℓ units yields an algebraic profit:

$$\psi_\ell(q_\ell, S_{t_\ell}) = q_\ell \cdot (S_{t_\ell} - K). \tag{2.9}$$

The price of the swing contract at time t_k with cumulative consumption Q is given by:

$$P_k(S_{t_k}, Q) = \operatorname{ess\,sup}_{(q_\ell)\in\mathcal{A}_{k,Q}^{Q_{\min},Q_{\max}}} \mathbb{E}\left[\sum_{\ell=k}^{n-1} \psi_\ell(q_\ell, S_{t_\ell}) \middle| \mathcal{F}_{t_k} \right], \tag{2.10}$$

with interest rates r_ℓ assumed to be zero.

2.2.2 Bang-Bang Property

When volume constraints are integers, and $Q_{\max} - Q_{\min}$ is a multiple of $q_{\max} - q_{\min}$, the optimal policy has a **bang-bang** structure, i.e.,

$$q_\ell \in \{A_\ell^-(Q_\ell), A_\ell^+(Q_\ell)\}$$

This means the best strategy is to always purchase either the minimum or the maximum allowed quantity. This significantly reduces computational complexity by limiting the number of scenarios.

3 Forward Price Dynamics

In energy markets, spot energy is not directly tradable due to storage constraints. As a result, market participants primarily transact using forward contracts.

To model pricing dynamics in this context, it is standard to consider the *forward price* $F_{t,T}$, which represents the price at time t of a contract delivering energy at a future time T.

Although the spot price S_t is not a tradable asset, it can still be modeled consistently within this framework by defining it as $S_t := F_{t,t}$.

3.1 One factor model

The one factor model used for forward price dynamics is:

$$\frac{dF_{t,T}}{F_{t,T}} = \sigma_F e^{-\lambda(T-t)} dW_t, \quad t \leq T, \tag{3.1}$$

where (W_t) is an $\mathcal{F}_t-$ Brownian motion.

The spot price is then given by the exponential martingale:

$$S_t = F_{0,t} \cdot \exp\left(\sigma_F X_t - \frac{1}{2}\Sigma_t^2\right), \tag{3.2}$$

with

$$X_t = \int_0^t e^{-\lambda(t-s)} dW_s, \quad \Sigma_t^2 = \frac{\sigma_F^2}{2\lambda}\left(1 - e^{-2\lambda t}\right). \tag{3.3}$$

an Ornstein-Uhlenbeck process and its variance.

3.2 Direct spot simulation in the one factor model

To simulate the spot process $(S_{t_k})_{0\leq k\leq n-1}$ given a time grid $(t_k)_{0\leq k\leq n-1}$, we first simulate $Y_{t_k} = e^{\lambda t_k} X_{t_k} = \int_0^{t_k} e^{\lambda s} dW_s$ which satisfies the recurrence relation :

$$Y_{t_{k+1}} = Y_{t_k} + \sigma_k Z^{[k+1]}, \quad \sigma_k^2 = \frac{e^{2\lambda t_{k+1}} - e^{2\lambda t_k}}{2\lambda} \tag{3.4}$$

where $(Z^{[k]})_{0\leq k\leq n-1}$ are i.i.d. $\mathcal{N}(0,1)$ variables. We then recover $(X_{t_k})_{0\leq k\leq n-1}$ with appropriate discounting and $(S_{t_k})_{0\leq k\leq n-1}$ according to (3.2).

3.3 Three factors model

The three factor model dynamics are given by:

$$\frac{dF_{i,T}}{F_{i,T}} = \sigma_{F,1} e^{-\lambda_1(T-t)} dW_t^1 + \sigma_{F,2} e^{-\lambda_2(T-t)} dW_t^2 + \sigma_{F,3} e^{-\lambda_3(T-t)} dW_t^3, \tag{3.5}$$

where for all $1 \leq i, j \leq 3$, the instantaneous correlation is given by:

$$\langle dW_t^i, dW_t^j \rangle = \begin{cases} dt & \text{if } i = j \\ \rho_{i,j} dt & \text{if } i \neq j \end{cases} \tag{3.6}$$

In this model, the spot price is given by:

$$S_t = F_{0,t} \cdot \exp\left(\sigma_F^\top X_t - \frac{1}{2}\Sigma_t^2\right), \tag{3.7}$$

where $\sigma_F = (\sigma_{F,1}, \sigma_{F,2}, \sigma_{F,3})^T$, $X_t = (X_t^1, X_t^2, X_t^3)^T$ and for all $1 \leq i \leq 3$,

$$X_t^i = \int_0^t e^{-\lambda_i(t-s)} dW_s^i \quad \text{and} \quad \Sigma_t^2 = \sum_{1 \leq i,j \leq 3} \rho_{i,j} \frac{\sigma_{F,i}\sigma_{F,j}}{\lambda_i + \lambda_j}(1 - e^{-(\lambda_i+\lambda_j)t}). \tag{3.8}$$

Simulations of the spot process in this model can be easily obtained in a similar way as in the one factor model.

4 Global optimization approach to swing contracts pricing

In this work, we solve the optimization problem (2.10) using a global optimization approach.

Since the optimal strategy is *bang-bang*. At each decision date t_k, and given a cumulative consumption Q_k, the control variable q_k can take values in a binary set: $\{A_k^-(Q_k), A_k^+(Q_k)\}$. Since q_k is assumed to be measurable with respect to the information set $\mathcal{F}_{t_k}$, there exists a measurable set $B_k \in \mathcal{F}_{t_k}$ such that:

$$q_k = A_k^-(Q_k) + \left(A_k^+(Q_k) - A_k^-(Q_k)\right) \mathbb{1}_{B_k}, \quad 0 \leq k \leq n-1. \tag{4.1}$$

To approximate the binary decision function $\mathbb{1}_{B_k}$, while allowing smooth optimization and gradient-based methods, we parametrize the control strategy q_k as:

$$q_k(S_k, Q_k^\theta, \theta) = A_k^-(Q_k^\theta) + \left(A_k^+(Q_k^\theta) - A_k^-(Q_k^\theta)\right) \sigma\left(\chi_k(S_{t_k}, Q_k^\theta, \theta)\right), \quad 0 \leq k \leq n-1, \tag{4.2}$$

where

$$\chi_k : \mathbb{R}^d \times \mathbb{R}_+ \times \Theta \to \mathbb{R}, \quad (S, Q, \theta) \mapsto \chi_k(S, Q, \theta),$$

,σ is the sigmoid function and Θ is a finite-dimensional parameter space to be determined.

The original optimization problem is then finite dimensional and reformulated as :

$$\theta^* \in \arg\max_{\theta \in \Theta} \mathbb{E}\left(\sum_{k=0}^{n-1} \psi_k(q_k(S_{t_k}, Q_k^\theta, \theta), S_{t_k})\right), \tag{4.3}$$

where ψ_k represents the profit function at time t_k.

4.1 PV strat

In this parametrization, the function $\chi_k(S, Q, \theta)$ is defined as a linear combination of the spot price S, a function $\eta(Q)$ of the quantity Q representing the normalized remaining purchase capacity, and a constant term:

$$\chi_k(S, Q, \theta) = \theta_{k,1}(S - K) + \theta_{k,2}\eta(Q) + \theta_{k,3}, \quad \text{with } \eta(Q) = \frac{Q - Q_{\min}}{Q_{\max} - Q_{\min}}. \tag{4.4}$$

Here, θ is a vector of real parameters of dimension $3N$, where N is the number of exercise dates.

This representation is an intuitive approach that models the optimal decision as resulting from a tradeoff between the potential immediate gain $S - K$ and the remaining purchase capacity $\eta(Q)$, and therefore, the long-term gain.

4.2 NN strat

In this second approach based on neural networks, we replace each $(\theta_{k,1}, \theta_{k,2}, \theta_{k,3}) \in \mathbb{R}^3$ of (5.4) by the output of a $\mathbb{R}^3$-valued feedforward neural network $\Phi : \mathbb{R}^3 \to \mathbb{R}^3$:

$$\chi_k(S, Q, \theta) = \langle \Phi(t_k, S - K, \eta(Q); \theta), I \rangle \,, \quad \text{with} \quad I = (S - K, \eta(Q), 1), \tag{4.5}$$

This modelling approach remains centered on the same idea, using the same prior knowledge of the optimal strategy. However, each contribution is modelled as output of a neural network. This approach enables the integration of non-linearity with respect to our inputs and enlarges the approximation class at the cost of a parameter space of dimension $\sum_{i=1}^{d} d_i d_{i+1} + d_d$. Notably, this dimension does not depend on the number of exercise dates.

5 Stochastic optimization algorithms

5.1 Adaptative Moment estimation (Adam)

Adam is an efficient first-order stochastic optimization method. In addition to standard Stochastic Gradient Descent (SGD), it incorporates the following key elements:

- **Momentum :** Maintains an exponentially decaying average of past gradients ($\mathbf{m}_t$). This helps to accelerate and stabilize learning, limiting oscillations.
- **Adaptive Learning Rates :** Adam adapts the learning rate for each parameter based on the second moments of the gradients ($\mathbf{v}_t$). Parameters that have received large gradients in the past have their learning rate reduced, while parameters with small gradients receive larger learning rates.
- **Bias Correction :** Because the initial moment estimates ($\mathbf{m}_0$ and $\mathbf{v}_0$) are initialized to zero, they are biased towards zero, especially during the initial timesteps. Adam includes a bias correction step ($\hat{\mathbf{m}}_t$ and $\hat{\mathbf{v}}_t$) to counteract this effect.

Using similar notations as presented in the course, the update is :

$$\begin{aligned} m_n &= (1 - \beta_1) \sum_{i=1}^{n} \beta_1^{n-i} H(\theta_n, Z^{[i]}) \quad \hat{m}_{n+1} = \frac{m_{n+1}}{1 - \beta_1^{n+1}} \\ v_n &= (1 - \beta_2) \sum_{i=1}^{n} \beta_2^{n-i} H(\theta_n, Z^{[i]})^2 \quad \hat{v}_{n+1} = \frac{v_{n+1}}{1 - \beta_2^{n+1}} \\ \theta_{n+1} &= \theta_n - \gamma \cdot \frac{\hat{m}_{n+1}}{\sqrt{\hat{v}_{n+1}} + \epsilon} \end{aligned} \tag{5.1}$$

where, γ is the stepsize, $\beta_1, \beta_2 \in [0, 1)$ and ϵ is a small positive constant.

Algorithm 1 Adam: A method for stochastic optimization

Require: γ (stepsize), $\beta_1, \beta_2 \in [0, 1)$ (decay rates), ϵ (small constant), initial parameters $\boldsymbol{\theta}_0$
Require: Initialize $\mathbf{m}_0 \Leftarrow \mathbf{0}$ (first moment), $\mathbf{v}_0 \Leftarrow \mathbf{0}$ (second moment)
Ensure: Optimized parameters $\boldsymbol{\theta}_t$
for $t = 1$ to convergence **do**
  $\mathbf{g}_t \Leftarrow \nabla_{\boldsymbol{\theta}_{t-1}} f_t(\boldsymbol{\theta}_{t-1})$ ▷ Compute gradient
  $\mathbf{m}_t \Leftarrow \beta_1 \cdot \mathbf{m}_{t-1} + (1 - \beta_1) \cdot \mathbf{g}_t$ ▷ Update first moment
  $\mathbf{v}_t \Leftarrow \beta_2 \cdot \mathbf{v}_{t-1} + (1 - \beta_2) \cdot (\mathbf{g}_t \odot \mathbf{g}_t)$ ▷ Update second moment
  $\hat{\mathbf{m}}_t \Leftarrow \mathbf{m}_t / (1 - \beta_1^t)$ ▷ Bias correction for first moment
  $\hat{\mathbf{v}}_t \Leftarrow \mathbf{v}_t / (1 - \beta_2^t)$ ▷ Bias correction for second moment
  $\Delta\boldsymbol{\theta}_t \Leftarrow -\gamma \cdot \hat{\mathbf{m}}_t / (\sqrt{\hat{\mathbf{v}}_t} + \epsilon)$ ▷ Compute update
  $\boldsymbol{\theta}_t \Leftarrow \boldsymbol{\theta}_{t-1} + \Delta\boldsymbol{\theta}_t$ ▷ Apply update
end for

5.2 Preconditioned Stochastic Gradient Langevin Dynamics (PSGLD)

Stochastic Gradient Langevin Dynamics (SGLD), also a first order algorithm, differs by adding Gaussian noise to each SGD update. This stochastic perturbation help escape local minima and provides convergence to an invariant distribution centered near $\arg\min J$. In PSGLD, we scale the update with a preconditioner based on moving averages of past gradients in similar fashion as in Adam, the update is :

$$\theta_{n+1} = \theta_n - \gamma P_{n+1} H(\theta_n, Z^{[n+1]}) + \sigma\sqrt{\gamma} P_{n+1}^{1/2} \eta_{n+1} \tag{5.2}$$

where $\eta_n \sim \mathcal{N}(0, I_d)$ are i.i.d. noise vectors and P_n are the diagonal preconditioners.

Algorithm 2 PSGLD Updating

Require: Step size γ, decay rate $\beta \in (0, 1)$, small constant λ, identity matrix Id_d
Require: Initialize parameter vector θ_0, second moment vector $MS_0 \Leftarrow 0$
Ensure: Optimal parameter θ^*
while θ_n not converged **do**
  $g_{n+1} \Leftarrow H(\theta_n, Z_{n+1})$ ▷ Compute stochastic gradient
  $MS_{n+1} \Leftarrow \beta \cdot MS_n + (1 - \beta) \cdot (g_{n+1} \odot g_{n+1})$ ▷ Update second moment
  $P_{n+1} \Leftarrow \text{diag}\left(Id_d \oslash (\lambda \cdot Id_d + \sqrt{MS_{n+1}})\right)$ ▷ Compute preconditioner
  $\theta_{n+1} \Leftarrow \theta_n - \gamma \cdot P_{n+1} \cdot g_{n+1} + \sqrt{\gamma} \cdot N(0, P_{n+1})$ ▷ Update parameters
end whilereturn θ_n

6 Numerical Results

In this section, we present the numerical experiments conducted to assess the performance of the two parametric strategies developed for swing contract pricing: the explicit Payoff-Volume strategy (PV strat) and the Neural Network strategy (NN strat).

We consider two contract settings, as in Lemaire et al. (2024):

- Case 1: a one-month swing contract with $n = 31$ exercise dates, minimum and maximum cumulative volumes $Q_{\min} = 140$ and $Q_{\max} = 200$, local volume bounds $q_{\min} = 0$ and $q_{\max} = 6$, and a strike price $K = 20$.
- Case 2: a one-year swing contract with $n = 365$ exercise dates, minimum and maximum cumulative volumes $Q_{\min} = 1300$ and $Q_{\max} = 1900$, local volume bounds $q_{\min} = 0$ and $q_{\max} = 6$, and a strike price $K = 20$.

Unless stated otherwise, the spot price dynamics follows a one-factor mean-reverting model with parameters $F_0 = 20, \lambda = 4$ and $\sigma = 0.7$.

All experiments were conducted on Google Colab with a Tesla T4 GPU and 16 GB of RAM.

Each iteration of the stochastic optimization algorithm uses a batch of 2^{14} simulated paths, and the final contract valuation is performed using 10^6 independent Monte Carlo paths. The learning rate is fixed to 0.1 throughout the experiments.

Due to time constraints, we did not replicate the train-evaluate process 100 times as in Lemaire et al. (2024). As a consequence, the reported confidence intervals only reflect the Monte Carlo evaluation error for a fixed learned strategy and do not capture the learning variability.

Throughout all experiments, we use standard variance reduction techniques to improve the precision of the Monte Carlo estimates. Namely, we apply control variates and antithetic sampling during the evaluation phase. Further details on the implementation of these techniques can be found in the accompanying Jupyter notebook.

Before presenting quantitative comparisons, we provide a visual illustration of the behavior of a learned control on a single trajectory. In the figure below, we display:

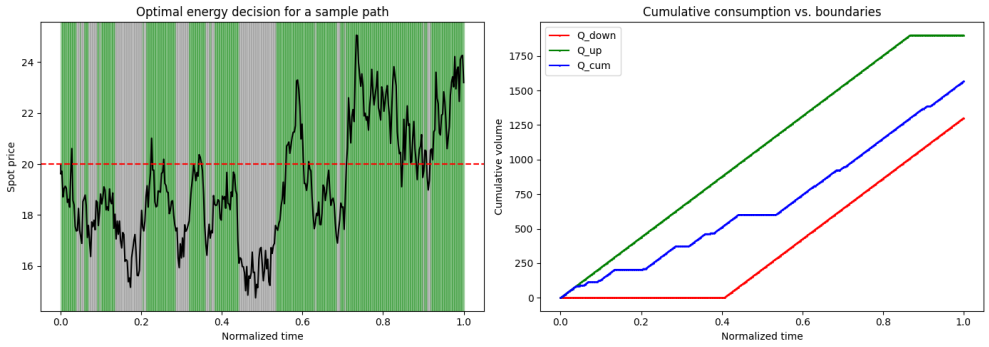

Fig. 1 On the left panel, the spot price evolution together with the exercise decisions. On the right panel, the cumulative consumption compared to the contract boundaries.

The decisions are evaluated using the learned strategy but enforcing an exact bang-bang control by replacing the logistic function with an indicator function. The green areas correspond to exercise at the maximum allowed quantity $q_{\max}$, while the grey areas correspond to exercise at the minimum quantity q_{min}.

We observe that the learned policy behaves reasonably well: consumption is typically avoided during low price periods, as illustrated by the absence of purchases during the downward peaks in the spot price trajectory. This qualitative behavior is consistent with the structure of the optimal control expected under bang-bang policies.

We now move on to a detailed comparison between the Adam and pSGLD optimization algorithms.

6.1 Adam vs pSGLD

Table 1 reports the results obtained using the Adam optimizer, for both PV strat and NN strat, and for two different numbers of training iterations ($N = 1000$ and $N = 3000$). As expected, increasing the number of iterations slightly improves the results, although the gains are marginal (e.g., from 63.31 to 63.43 for PV strat). The computational time, however, increases significantly, from 55 to 158 seconds for PV strat, and from 73 to 200 seconds for NN strat. This confirms that NN strat is more computationally demanding than PV strat.

Interestingly, the prices obtained using NN strat are consistently higher and slightly closer to the forced bang-bang evaluations (P_0^{BG}). This suggests that the NN parameterization is more expressive and better captures the near-optimal control, albeit at a higher computational cost.

Table 2 presents the results obtained using the pSGLD optimizer for a fixed number of iterations $N = 1000$, and for several values of the noise level σ and the friction parameter β. For PV strat, pSGLD performs reasonably well, with prices close to those obtained by Adam and slightly lower variance in computational time. However, for NN strat, pSGLD yields significantly lower prices, especially when $\sigma = 10^{-5}$, indicating a potential underfitting due to excessive noise in the updates.

Overall, in our experiments—which are significantly less extensive than those in Lemaire et al. (2024)—Adam appears to provide better performance, especially for NN strat. Nevertheless, to maintain consistency with the experimental setup of the original paper, we adopt pSGLD with $\sigma = 10^{-6}$ and $\beta = 0.8$ for the rest of this report.

N	Method	P_0	P_0^{BG}	Time (s)
3000	PV	63.43 ([63.36, 63.5])	63.47 ([63.4, 63.54])	158.58
1000	PV	63.31 ([63.24, 63.38])	63.44 ([63.37, 63.52])	55.66
3000	NN	63.60 ([63.53, 63.67])	63.56 ([63.49,63.63])	199.93
1000	NN	63.62 ([63.55,63.69])	63.61 ([63.54,63.68])	73.32

Table 1 Results using Adam optimization algorithm. A confidence interval (95%) is in brackets. Column "Time" includes both training and valuation times.

σ	β	Method	P_0	P_0^{BG}	Time (s)
$1 \cdot e^{-6}$	0.8	PV	63.2 ([63.12, 63.27])	63.32 ([63.25, 63.4])	53.95
$1 \cdot e^{-6}$	0.9	PV	63.15 ([63.08, 63.22])	63.32 ([63.25, 63.40])	53.44
$1 \cdot e^{-5}$	0.9	PV	63.18 ([63.11, 63.25])	63.45 ([63.38,63.52])	54.16
$1 \cdot e^{-6}$	0.8	NN	63.34 ([63.27, 63.41])	63.28 ([63.21, 63.35])	69.06
$1 \cdot e^{-6}$	0.9	NN	62.96 ([62.90, 63.02])	62.96 ([63.9, 63.02])	73.15
$1 \cdot e^{-5}$	0.9	NN	61.58 ([61.49, 61.67])	61.58 ([61.49,61,67])	71.37

Table 2 Results using pSGLD optimization algorithm for N=1000. A confidence interval (95%) is in brackets. Column "Time" includes both training and valuation times.

6.2 Transfer Learning

Transfer learning consists in using parameters learned from a related source task to initialise training on a more complex target task. Following Lemaire et al. (2024), we study the impact of this approach on training time and robustness.

6.2.1 Speed up learning

We evaluate the effect of transfer learning on Case 2 (one-year contract with daily exercise). Parameters are first learned on an aggregated version of the contract, where one exercise is allowed per month and local constraints are scaled accordingly ($q_{\max} = 180$). In this setting, the number of decision dates is reduced to 12. For NN strat, the transfer is direct. For PV strat, each monthly parameter is copied to all corresponding days of the month in the full-resolution contract.

Without transfer, we train the model for 1000 iterations on the full contract. With transfer, we perform 500 iterations on the aggregated contract, followed by 300 iterations on the full one. Table 3 shows that transfer learning reduces training time

significantly, with little impact on the final price. The value T_{agg} corresponds to the pre-training time on the aggregated contract and is not included in T_{train}.

Transfer Learning	Method	P_0	P_0^{BG}	T_{agg}	$(T_{\text{train}}, T_{\text{eval}})$
No	PV	2654.84 ([2653.04, 2656.63])	2677.85 ([2676.06, 2679.65])	0	(620.56, 79.2)
Yes	PV	2655.02 ([2653.23, 2656.81])	2673.56 ([2671.78, 2675.35])	18.07	(187.65, 74.34)
No	NN	2654.52 ([2652.72, 2656.31])	2654.52 ([2652.72, 2656.31])	0	(782.95, 32.19)
Yes	NN	2624.81 ([2623.0, 2626.61])	2624.80 ([2623.0, 2626.61])	29.26	(235.88, 32.86)

Table 3 Effect of transfer learning on performance and training time in Case 2. Prices are given with 95% confidence intervals. T_{agg} is the pre-training time (in seconds) on the aggregated contract.

6.2.2 Shifts in the market

We now study the effect of moderate market shifts on the robustness of pre-trained strategies and the relevance of re-training. Following Lemaire et al. (2024), we consider three perturbed scenarios starting from Case 2 (one-year contract with daily exercise), each modifying only one component of the environment:

- M_1: shift in initial forward price, from $F_0 = 20$ to $F_0 = 22$;
- M_2: shift in global volume constraints, from $[1300, 1900]$ to $[1400, 2000]$;
- M_3: shift in strike, from $K = 20$ to $K = 18$.

For each market configuration, we compare two approaches:

1. *Reuse*: directly reuse the strategy trained on the reference model;
2. *Retrain*: continue training the same model for 300 additional iterations, starting from the current parameters.

Results are reported in Table 4. Across all configurations, directly reusing the original strategy yields reasonable prices, indicating that both PV strat and NN strat are relatively robust to small changes in market conditions.

However, retraining systematically improves performance. The gain is most pronounced in M_3, where the drop in strike significantly alters the optimal exercise profile, making retraining particularly valuable. In M_1 and M_2, improvements remain modest, suggesting that the existing strategy still adapts well to those perturbations.

These results support the idea that transfer learning is a practical and efficient tool: with only a few additional iterations, it allows the strategy to quickly adapt and recover most of the value lost due to market changes.

6.3 Delta

We focus here on the sensitivity of the swing contract price to the initial forward prices.

Figure 2 displays the deltas computed for both PV strat and NN strat. The left panel corresponds to Case 1 (monthly contract), trained for 1000 iterations. The right panel corresponds to Case 2 (annual contract), with transfer learning (500 iterations on the aggregated version and 300 on the full contract).

Case	Method	Reuse	Retrain	$(T_{\text{train}}, T_{\text{eval}})$
M_1	PV	3717.03 ([3715.04, 3719.03])	3789.02 ([3787.12, 3790.9])	(172.17, 77.49)
M_2	PV	2476.66 ([2474.99, 2478.32])	2499.99 ([2498.34, 2501.63])	(172.46, 77.85)
M_3	PV	3432.22 ([3430.4, 3434.03])	3534.21 ([3532.5, 3535.91])	(171.98, 76.51)
M_1	NN	3748.25 ([3746.25, 3750.24])	3766.17 ([3764.26, 3768.07])	(228.38, 33.36)
M_2	NN	2434.27 ([2432.61, 2435.93])	2441.69 ([2439.87, 2443.5])	(229.41, 33.44)
M_3	NN	3477.4 ([3475.58, 3479.23])	3503.22 ([3501.51, 3504.94])	(229.57, 32.8)

Table 4 Robustness of pre-trained strategies to market shifts and impact of retraining. Results are shown for three market variations (M_1 to M_3), using PV strat and NN strat. Reuse refers to direct application of the pre-trained strategy, while retrain involves 300 additional iterations. Confidence intervals are given at 95%.

The delta profiles obtained from the two parameterizations exhibit broadly similar shapes in both cases. However, the NN strat consistently produces smoother curves, while the PV strat deltas appear somewhat more erratic, especially noticeable in Case 2 (right panel).

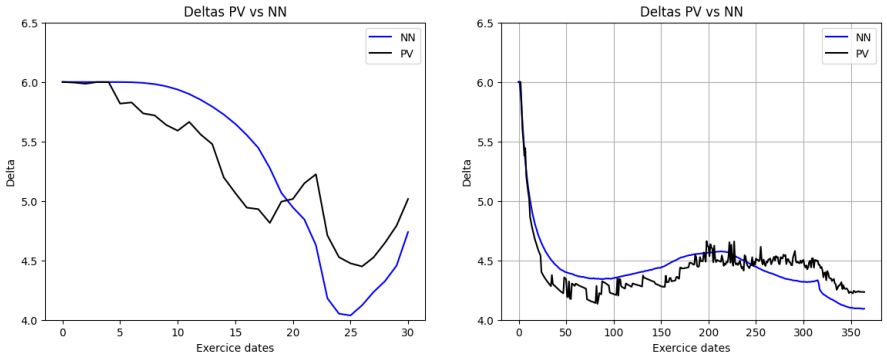

Fig. 2 Delta comparison between PV strat (black) and NN strat (blue). Left: Case 1 (1000 training iterations). Right: Case 2 (500 + 300 iterations with transfer learning).

6.4 Three factor model

We now test the methods on Case 1 under a three-factor forward model with correlated latent processes. We use a constant correlation matrix with ones on the diagonal and a uniform off-diagonal coefficient $\rho \in \{0.6, 0.3, -0.2\}$.

In addition to the standard setup, we also test a variant where the latent state vector (X_t^1, X_t^2, X_t^3) is included in the input of the neural network.

Tables 5 and 6 show the results. As expected, the option price decreases as the correlation ρ decreases. This reflects a lower synchronisation between the latent factors, leading to less favourable scenarios for exercising the option.

The gap between P_0 and P_0^{BG} is consistently very small, indicating that the learned strategies are close to bang-bang optimality across all settings.

Finally, including the latent state in the network input has no significant impact on pricing performance. This suggests that the network architecture already captures the relevant features of the system through its time encoding and input structure.

ρ	Method	P_0	P_0^{BG}	Time (s)
0.6	PV	167.59 ([167.28, 167.89])	168.12 ([167.82, 168.43])	108.83
0.3	PV	143.49 ([143.25, 143.74])	143.77 ([143.52, 144.01])	103.01
-0.2	PV	88.38 ([88.25, 88.5])	88.42 ([88.29, 88.55])	92.53
0.6	NN	168.05 ([167.74, 168.35])	168.03 ([167.74, 168.35])	128.94
0.3	NN	141.15 ([140.88, 141.42])	141.15 ([140.87, 141.42])	127.73
-0.2	NN	88.16 ([88.05, 88.27])	88.16 ([88.05, 88.27])	128.41

Table 5 Swing option prices under the three-factor model for different correlation values.

ρ	P_0	P_0^{BG}	Time (s)
0.6	167.90 ([167.59, 168.21])	167.90 ([167.59, 168.21])	131.33
0.3	143.43 ([143.17, 143.68])	143.43 ([143.17, 143.68])	132.20
-0.2	87.36 ([87.23, 87.5])	87.36 ([87.23, 87.5])	130.65

Table 6 Neural network with latent state (X_t^1, X_t^2, X_t^3) as input.

References

Lemaire, V., Pagès, G., Yeo, C. (2024, 1). Swing contract pricing: With and without neural networks. *Frontiers of Mathematical Finance*, *3*(2), 270–303, https://doi.org/10.3934/fmf.2024007